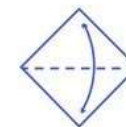
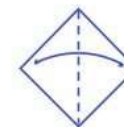
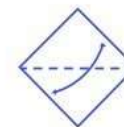
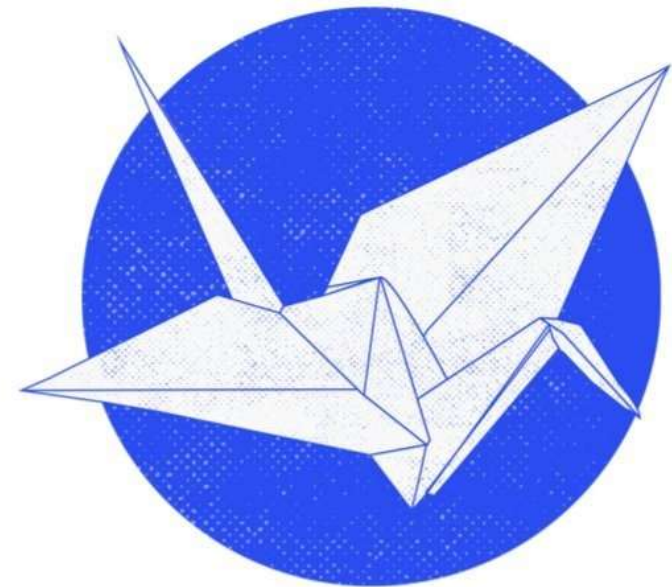


Computational Origami

- Shenglun Hutton



O
R
I
G
A
M
I

Aims of the project

optimize m subject to $\begin{cases} |v_i - v_j| \geq ml_{i,j} \text{ for all } i, j \\ v_i \in P' \text{ for all } i \end{cases}$.

$$l_{i,j} \equiv \sum_{e_k \in P_{i,j}} w(e_k).$$

```
1110
1111 update_graph();
```

Understand the maths
behind origami



Create a program that
generates origami
crease patterns



To make a cool origami
dragon
(disclaimer: it didn't
happen)

Lemma 4.1 Consider an equilateral triangle $\Delta = (v_1, v_2, v_3)$ bounded by the lines ℓ_1, ℓ_2, ℓ_3 through the triangle edges e_1, e_2, e_3 . For an arbitrary point p , let d_j be the distance of p from ℓ_j . Define $y_j = d_j$, if p is on the same side of ℓ_j as Δ , and $y_j = -d_j$ if p is separated from Δ by ℓ_j . Then $\sum_{j=1}^3 y_j$ is independent of the position of p .

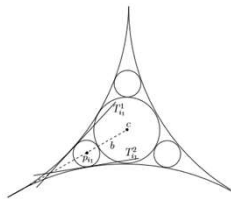
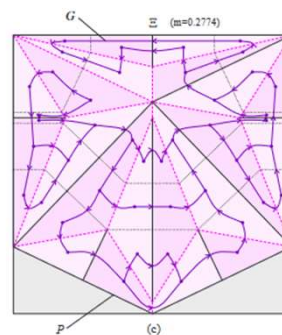
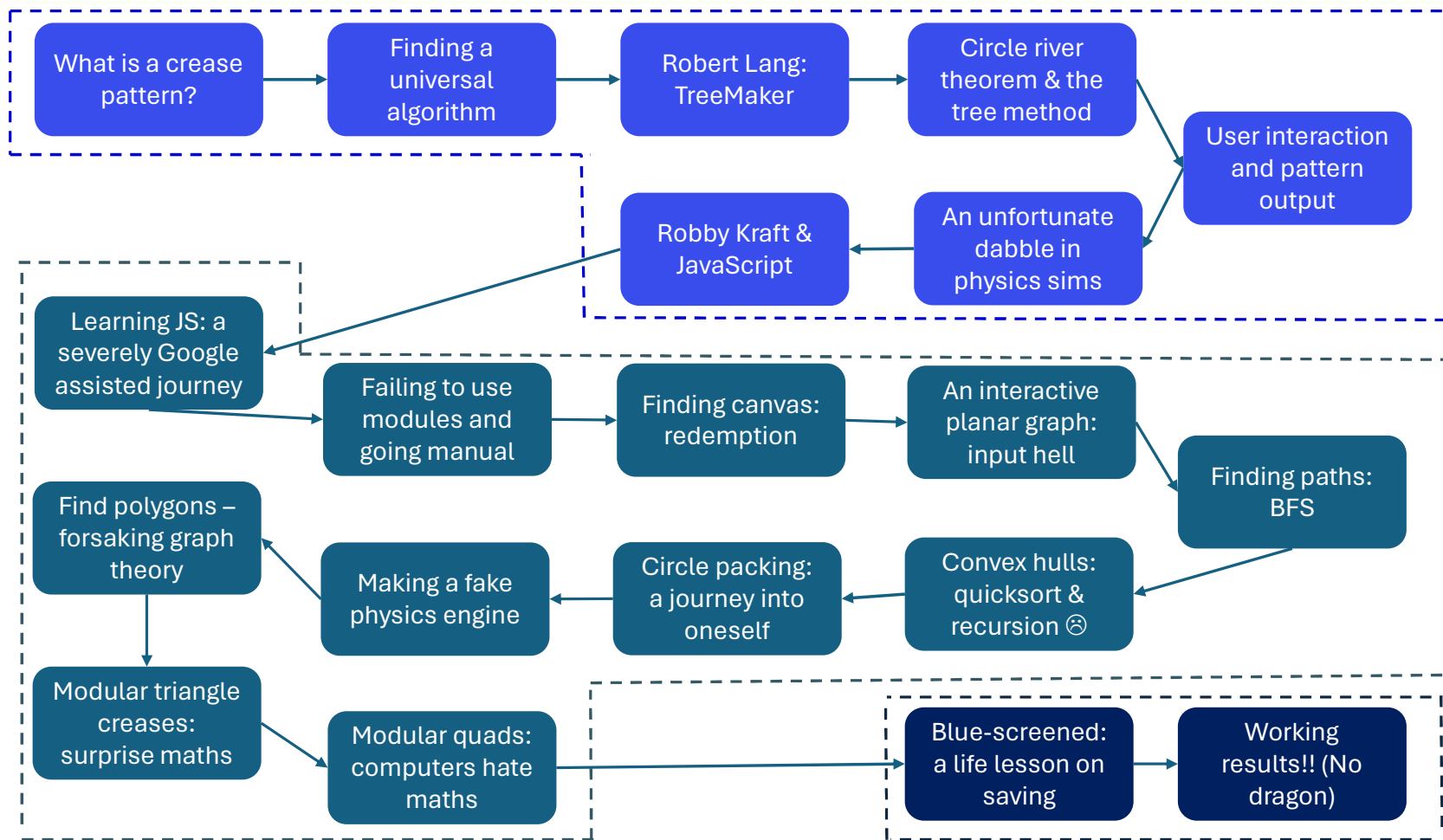


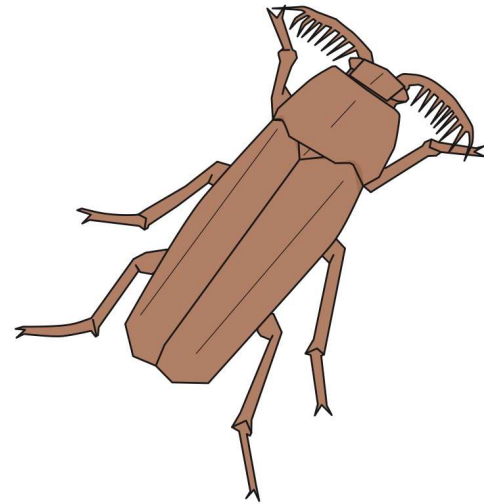
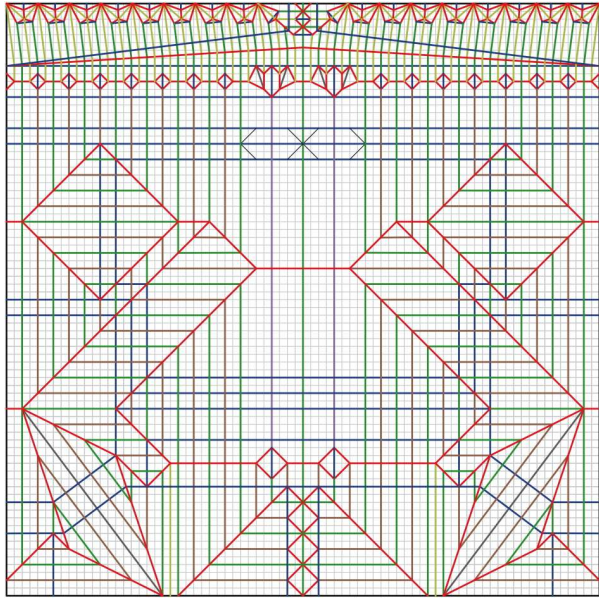
Figure 5: Changing the size of a shim.





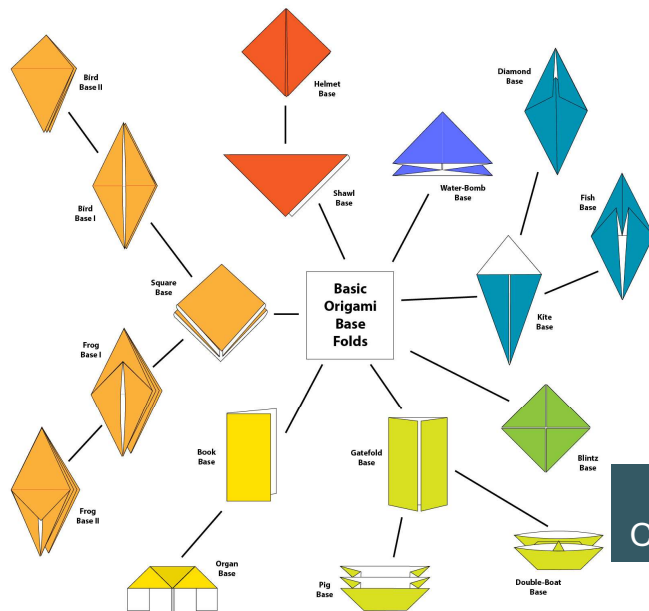
Origami foundations

Crease patterns

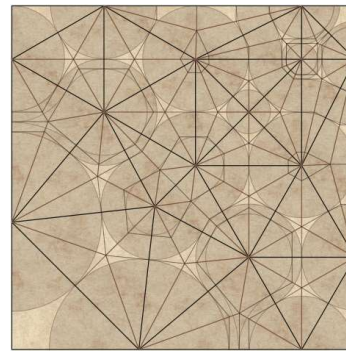


Origami foundations

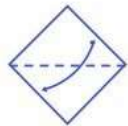
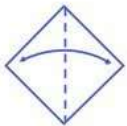
Traditional Bases



Computer generated bases



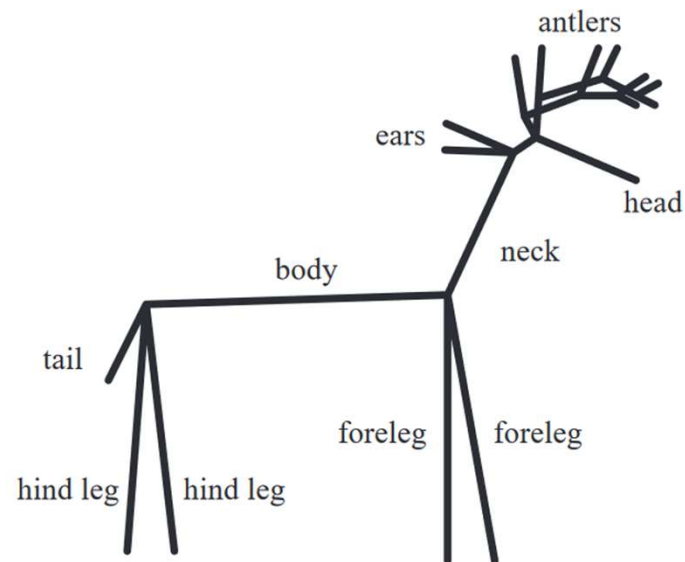
Flaps &
Connectors!



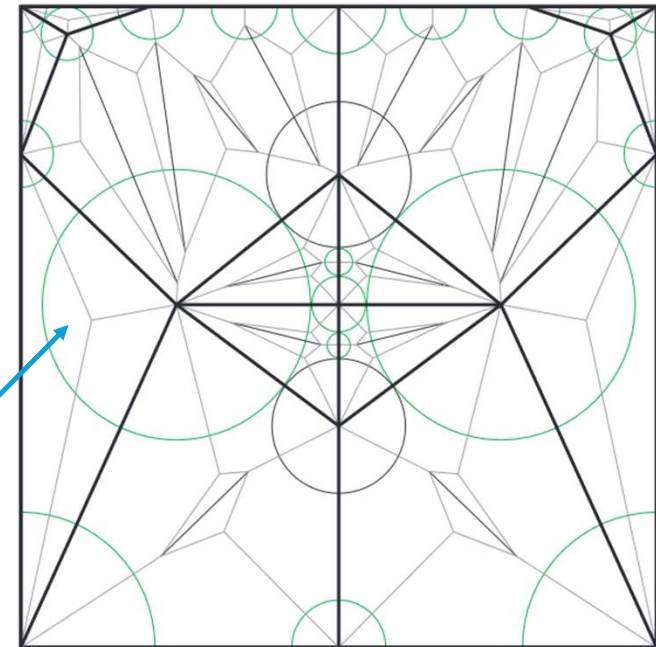
O R I G A M I

Robert Lang's tree method

Input: stick figure

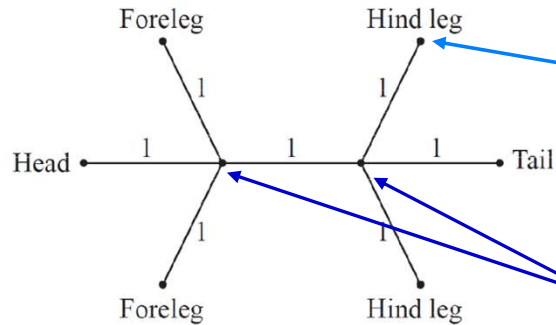
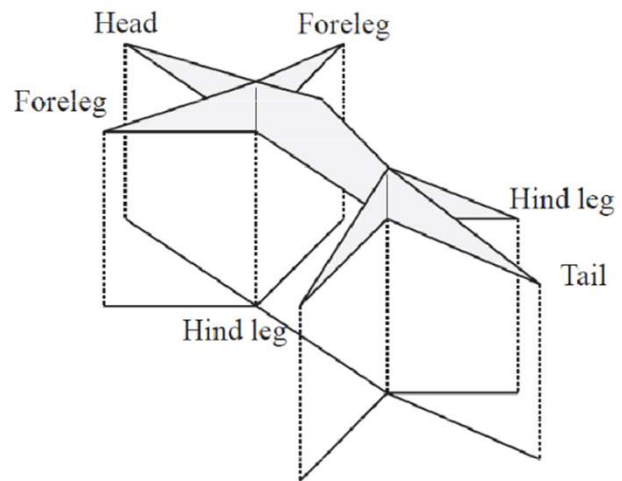


Output: base crease pattern



but why the circles?

Robert Lang's tree method

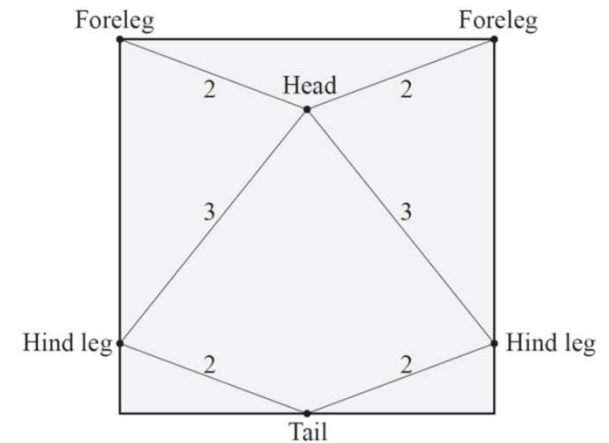


Mapping the nodes onto a square:

For a *uniaxial base*, tree is a projection

Internal nodes: represents multiple points on the square

Terminal nodes: only maps to one unique point on square



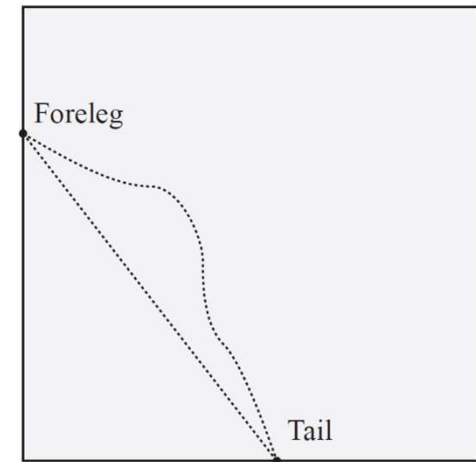
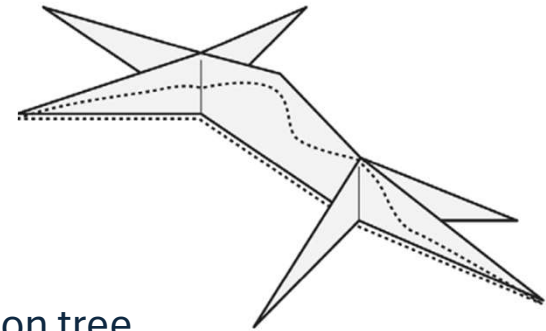
terminal nodes

internal nodes

Robert Lang's tree method

Active paths

- Shortest distance between two nodes = sum of edge lengths of shortest path on the tree
- (1) Distance between two nodes on square $\geq$ shortest path on tree
- Active path if distance equals path length on tree
- Active paths are major creases, they always exist
- If (1) holds for all *terminal nodes*, a crease pattern exists



Robert Lang's tree method

Theorem 1: For any two points P_1 and P_2 on the crease pattern, the length of path $L(P_1, P_2)$ is at least the length of the transformed path $G(F(L))$ in the embedded tree.

Proof: Consider such a path L , which in S consists of a collinear set of line segments l_i , one for each facet crossed by the path. The length of this path is given by $\sum l_i$. Upon transformation into the base, the path is transformed into a piecewise linear line $F(L)$ of the same length $\sum l_i$ since, within each facet, F preserves distances. However, upon taking the projection of the transformed path $G(F(L))$, each segment will be reduced in length by a factor $\cos \alpha_i$, where α_i is the angle between the transformed segment and the xy plane. Therefore, the length of the projection of the path is $\sum l_i \cos \alpha_i$. Since all of the multipliers are 1 or less, the length of the projection must be less than or equal to the length of the original path.

Corollary 1: If P_1 and P_2 are two points in the crease pattern S and $F(P_1)$ lies in the xy plane, the length of the path $L(P_1, P_2)$ is equal to the length of $G(F(L))$ if and only if $F(L)$ lies in the xy plane.

Proof: By theorem 1, equality holds only when each of the cosines is equal to 1 and the angles α_i are zero. Therefore each of the line segments of the transformed path is parallel to the xy plane. Since one endpoint of the path transforms to the xy plane and the path is parallel to the xy plane, the entire path must lie in the xy plane. Conversely, if the transformation lies in the xy plane, then all of the angles α_i are zero by definition.

Definition: A **vertex** is a point in the unit square where two or more creases come together.

Definition: A **terminal vertex** is a vertex whose transformation lies in the xy plane and whose projection is a terminal node of the tree graph.

Lemma 1. The length of the projection of a path between two terminal vertices is greater than or equal to the sum of the lengths of the edges between the two corresponding terminal nodes on the tree graph.

Proof: Since the tree graph is simply connected, the projection of the path must at a minimum include the edges between the two corresponding terminal nodes. Consequently, its length is greater than or equal to the sum of the lengths of the edges.

These definitions and results lead to the fundamental theorem underlying the algorithm of this paper:

Theorem 2: For any uniaxial base, the distance between any two terminal vertices in the crease pattern is greater than or equal to the sum of the lengths of the edges connecting the two corresponding terminal nodes in the tree graph.

Proof: Consider the path between two terminal vertices. According to theorem 1, the length of the projection of the path is less than or equal to the length of the path, which is the distance between the two vertices. But since the projection of the path must lie on the tree graph, and the tree graph is simply connected, the edges that lie between the corresponding terminal nodes must be a subset of the edges that comprise the projection of the path. Consequently, the sum of the lengths of the connecting edges is less than or equal to the length of the projection, which is less than or equal to the length of the original path.

Theorem 3. Every active path is either a crease or the edge of the square.

Proof: According to corollary 1 above, since the length of the active path equals the length of its projection, the transformed active path must lie in the xy plane. But since facets touch the xy plane only at their boundaries, the active path must lie along facet

boundaries. Since the boundaries of facets are either creases or the edge of the square, every active path must be composed of creases or the edge of the square. Since both paths and the edges of the square are straight lines, if any portion of a path is the edge of the square, the entire path must be the edge of the square. Consequently, the path is either wholly crease or wholly square edge.

Thus, given a tree graph, one can begin to design a crease pattern that folds into the corresponding uniaxial base by identifying a set of terminal vertices that satisfy the tree conditions; then the active paths between terminal vertices are a subset of the creases of the base. It remains to identify the remaining creases.

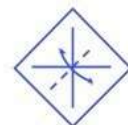
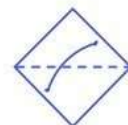
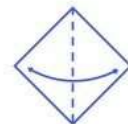
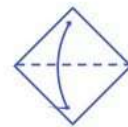
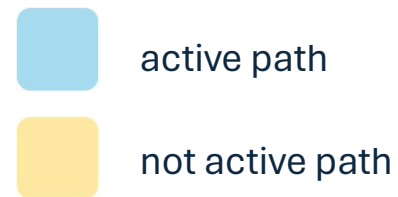
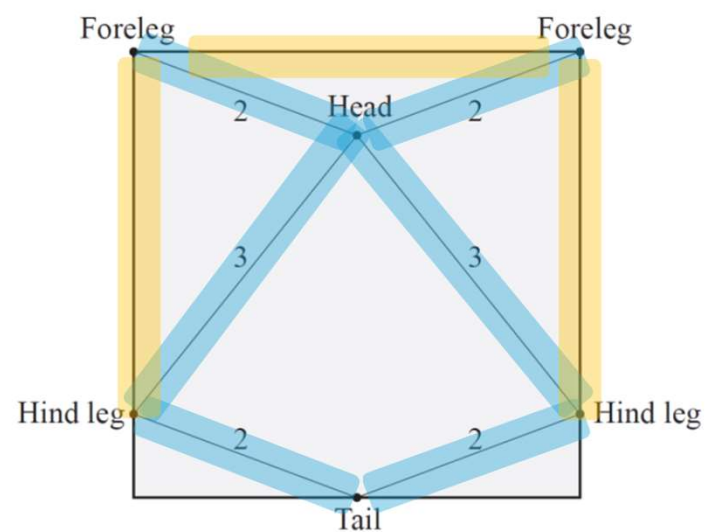
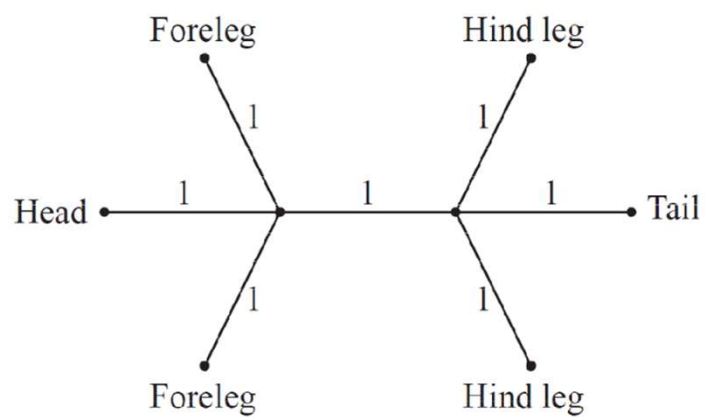
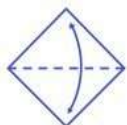
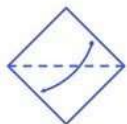
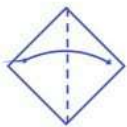
It can be shown that active paths cannot cross; they touch only at terminal vertices. Thus, active paths do more than define creases; they break up the unit square into regions that may be investigated independently.

Definition: an **active polygon** is a region of the unit square containing no terminal vertices in its interior whose boundary is formed exclusively by active paths.

Definition: a **subbase** is the transformation of an active polygon into a portion of the base.

Definition: a **subtree** is the projection of a subbase.

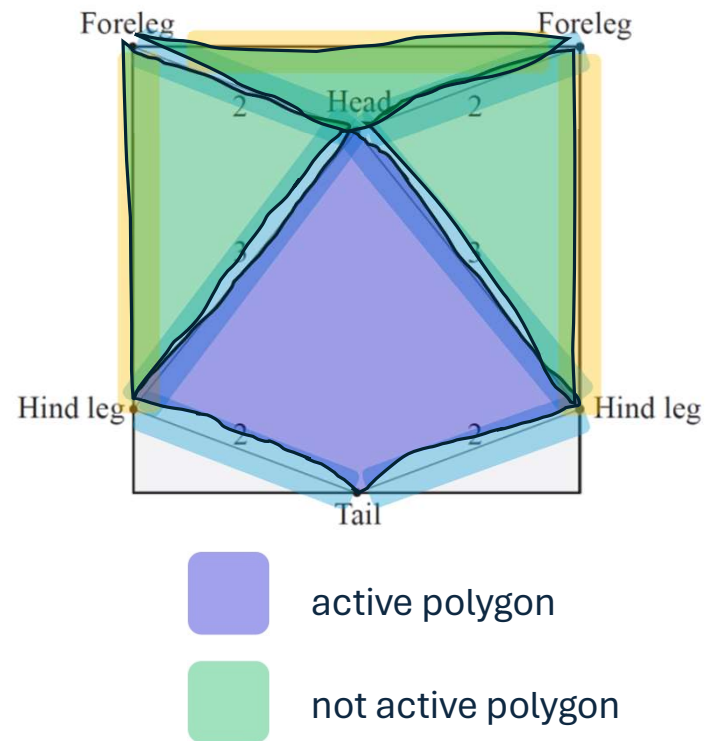
Observe that if every terminal vertex lies on the boundary of at least one active polygon, then the union of all of the subtrees corresponding to active polygons is the complete tree graph. Also note that the edges of an active polygon, being active paths, must lie in the xy plane after transformation. Thus, if one can identify the creases within each active polygon that transform it into a subbase, the individual subbases can figuratively be stitched together at their edges (the active paths) to form the overall base whose projection is the desired tree graph.



Robert Lang's tree method

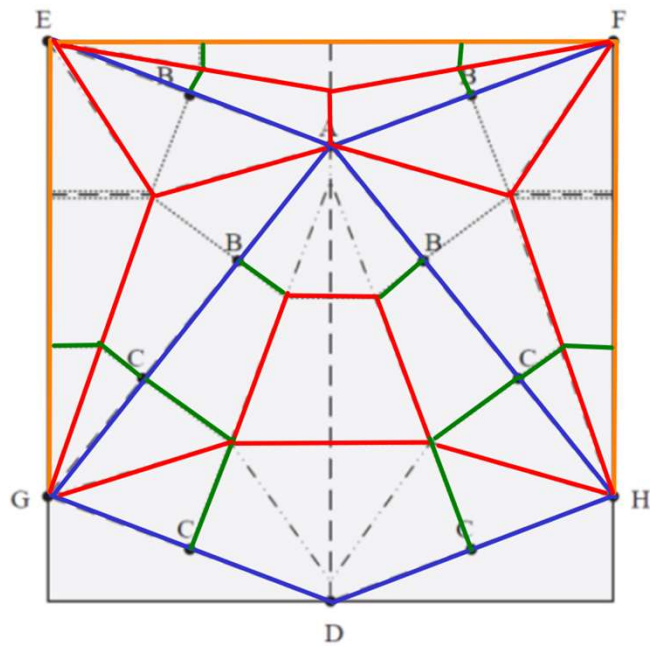
Active polygons

- Polygons where there's a cycle in the planar graph
- Active polygon if all edges are active paths
- If shared side is active, *sub-bases* are compatible
- Could cut out polygons, fold *sub-base*, glue together

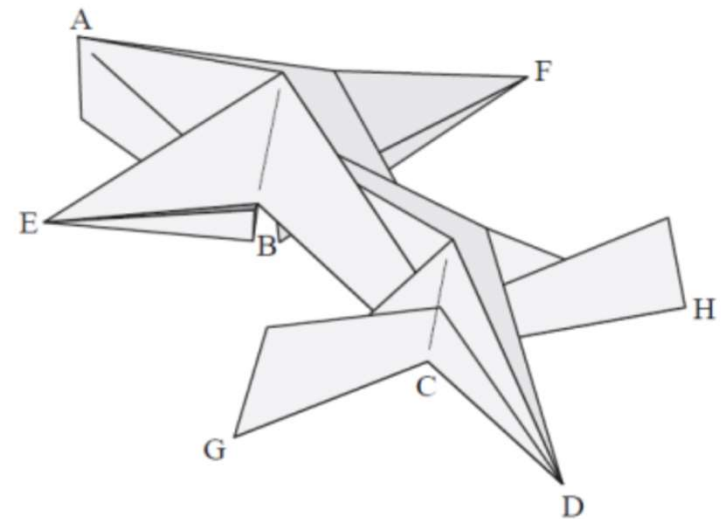


Robert Lang's tree method

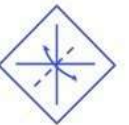
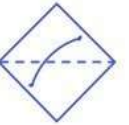
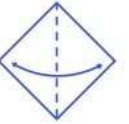
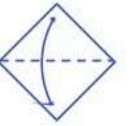
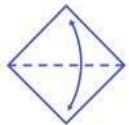
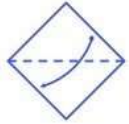
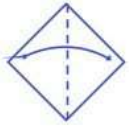
Final crease pattern



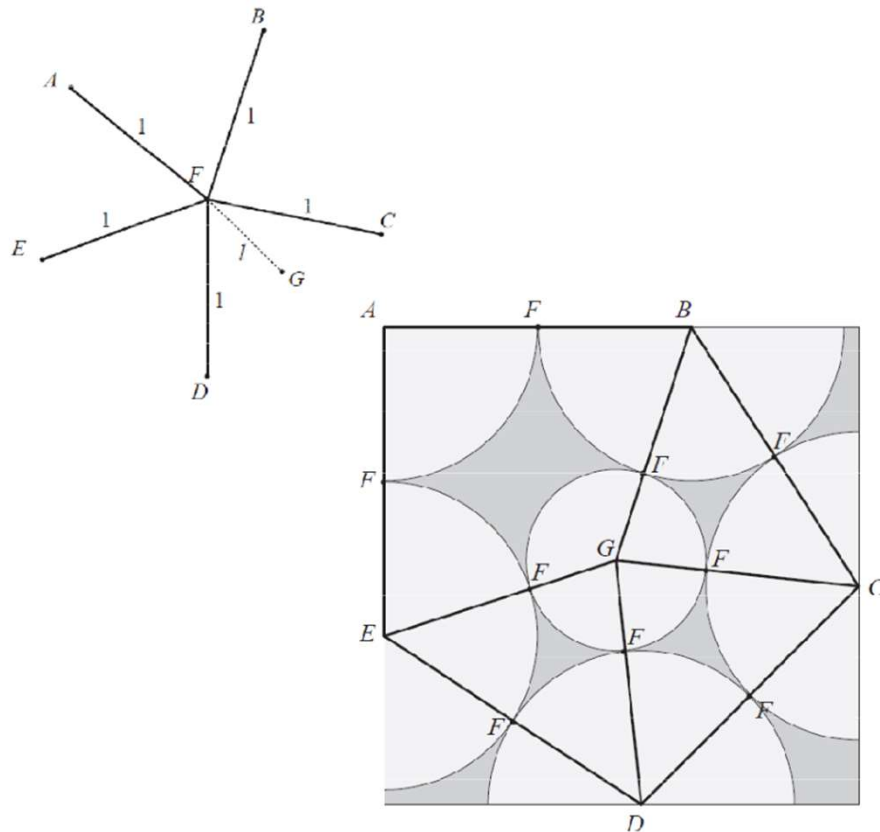
Folded base



But what about the circles?



Robert Lang's tree method

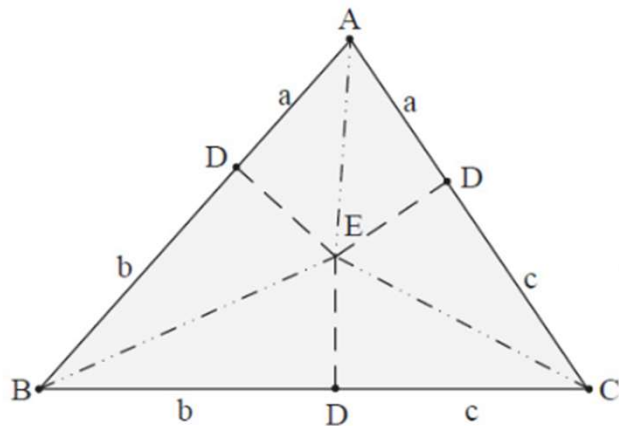


Circle packing to find active paths

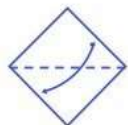
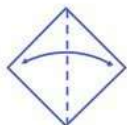
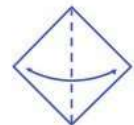
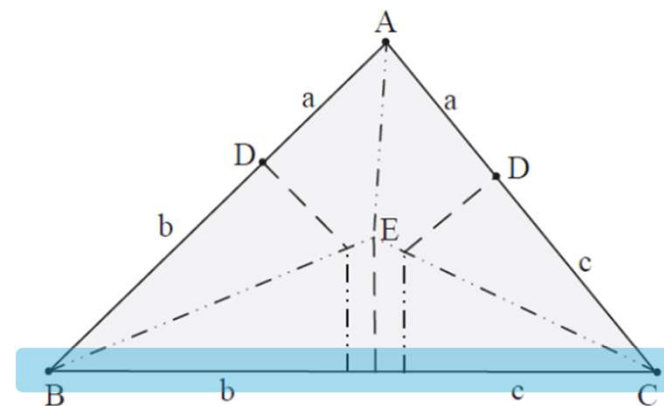
- Polygons greater than 4 are increasingly complex
- Each terminal node is the centre of a circle
- The circle radius is the length of its connecting edge
- Two circles tangent is the same as an active path existing
- Can use circle packing algorithms instead of complicated maths

Types of triangle molecules

3 Active Paths



2 active paths

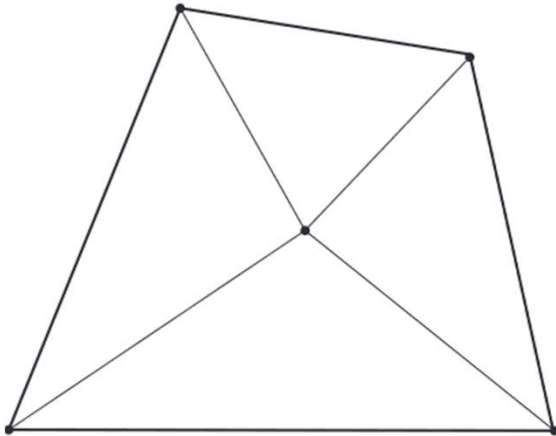


ORIGAMI

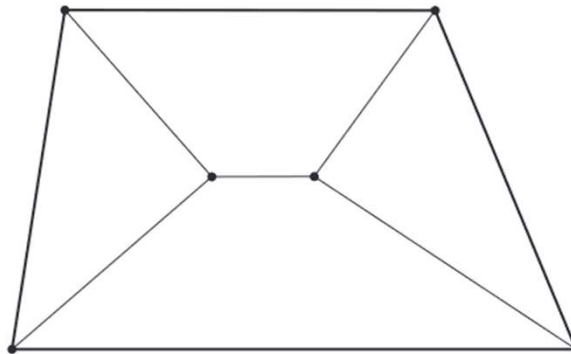


Types of quadrilateral molecules

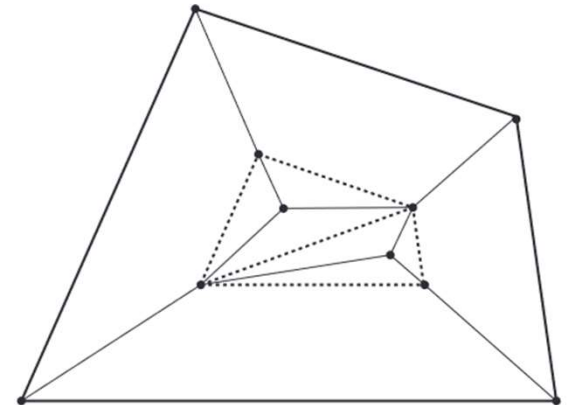
Husimi molecule



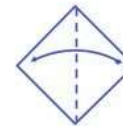
Maekawa molecule

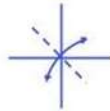


Lang's gusset quad molecule

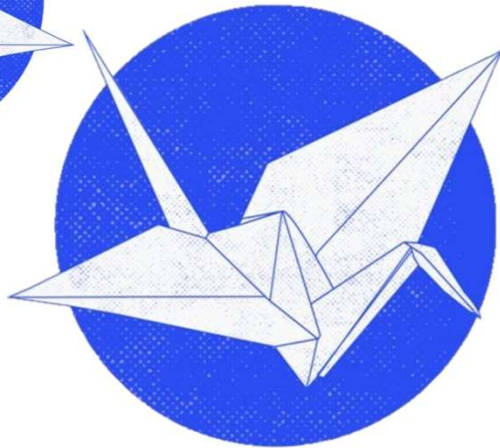
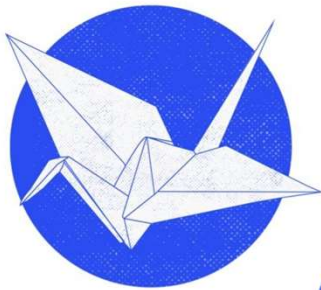


ORIGAMI

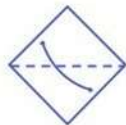
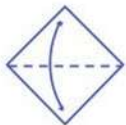




O
R
I
G
A
M
I

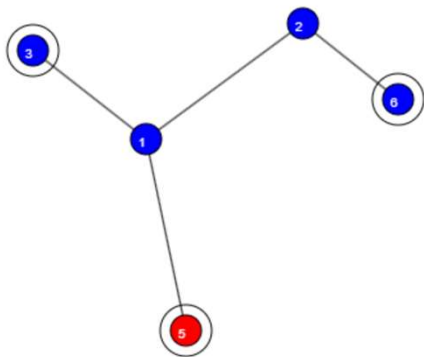


Implementation



Implementation: Initial interactive graph

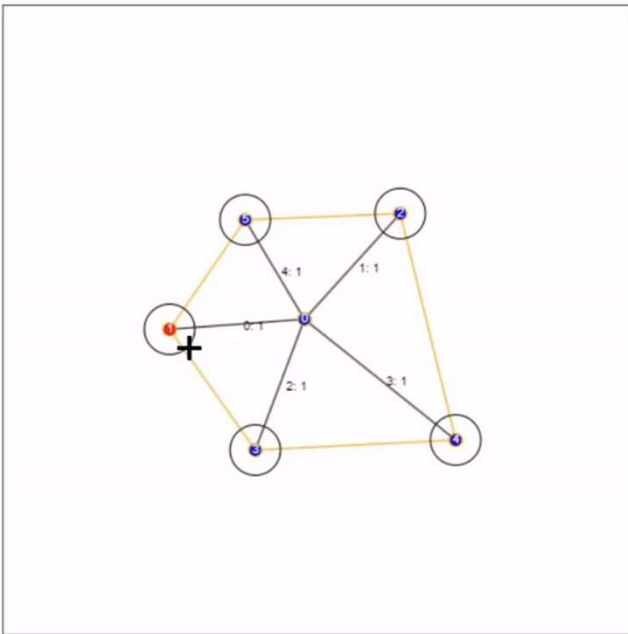
File: Test2.html



- **Actions:**
 - Left click: add/select node
 - Right click: delete node
 - Drag: move node
- **Passive:**
 - Terminal nodes, “vertices”, circled
 - Currently selected node is red
- **Data:**
 - Node objects saved with properties:
 - *x, y coordinates*
 - *Unique numerical ID*
 - *Adjacent nodes*

Implementation: Convex hull

File: Test5.html

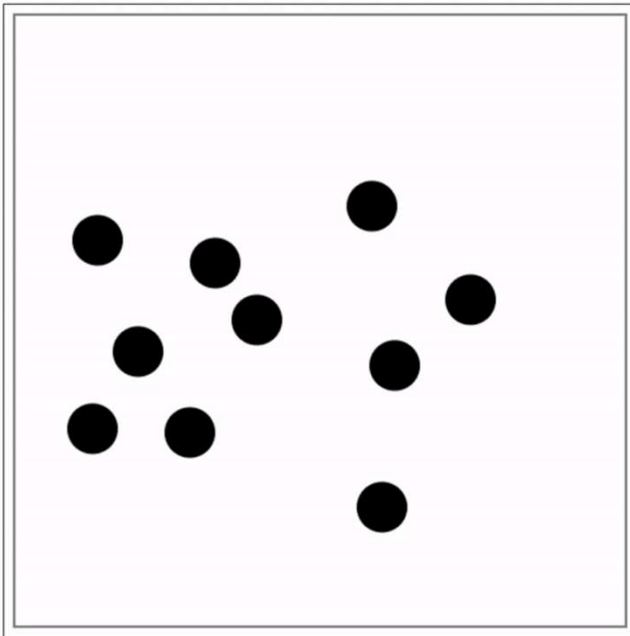


QuickSort method:

1. Takes initial furthest points A, B in the graph and constructs a line AB
2. Split graph into two sides $S1$ and $S2$
3. Finds furthest point C from line AB on side $S1$, replace line with two new lines AC and BC
4. Call step 2-3 recursively until there's no more points on the side, and add the line to the hull array
5. Do steps 2-4 but with the other sides aswell

Implementation: Circle packing

**Files: index.html
origami.js**

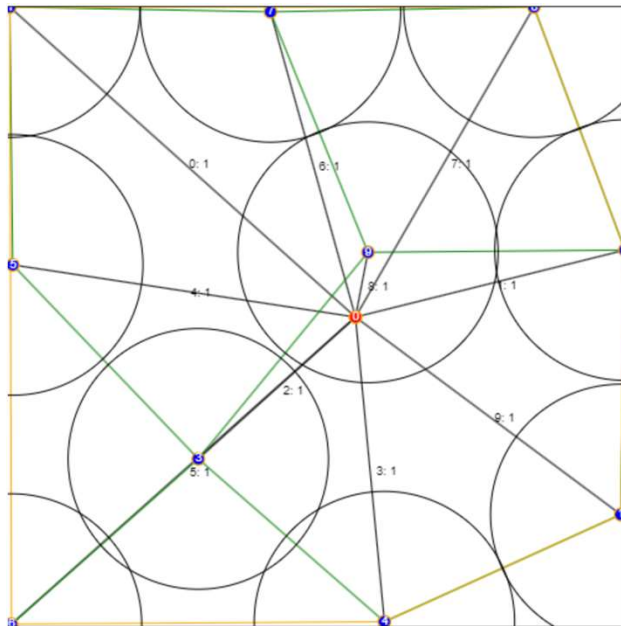


Trial & Many Errors of polygon packing:

- Initially tried to use pure maths – worst programming mistake
- Method:
 - Shrink square by 1px until condition (1) met
 - Every shrink, check for any overlaps/circles outside square – incredibly suboptimal
 - (1): tried to reposition circles to prevent overlaps 2000+ iterations without success
- Adding a new canvas for visualising the circle packing method

Implementation: Circle Packing

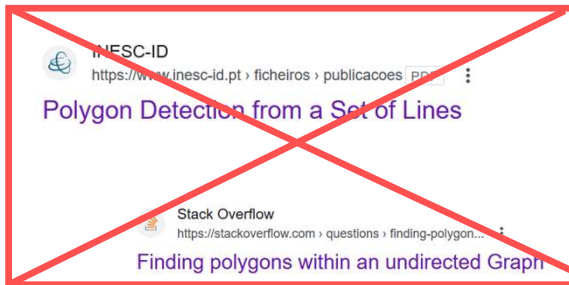
**Files: index.html
origami.js**



Results of packing – scale_everything():

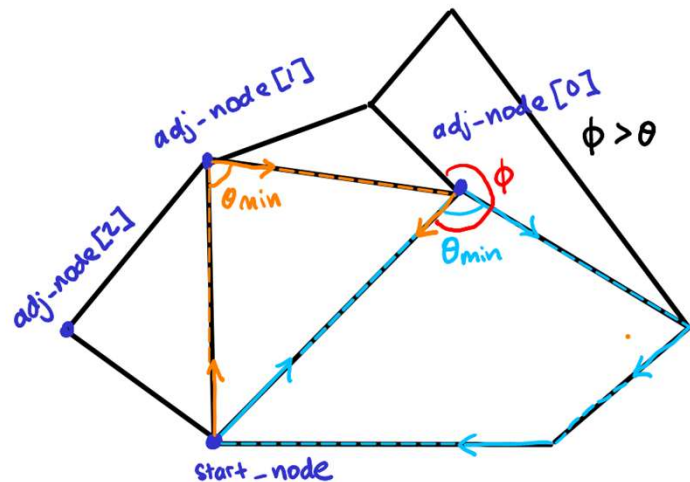
- Takes circle packing and scales it to canvas size
- A lot of scale factors to deal with:
 - backend working with arbitrary edge length radii, e.g. 1, 2
 - frontend working with pixels so visual scale factor taken into account

Implementation: Finding polygons

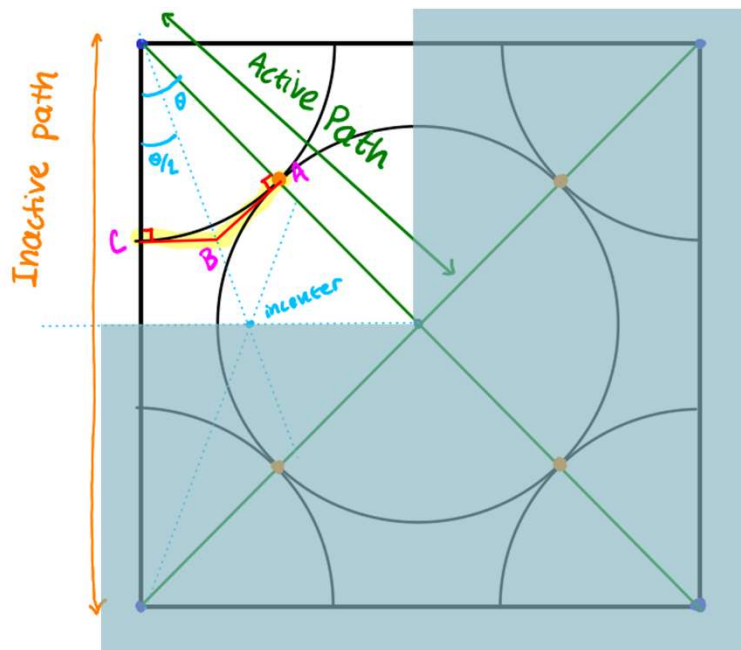


Finding polygons – is much harder than it seems:

- The proper way: Using DFS to find the minimum cycles in a planar graph
- The better way:
 - Looping through every node
 - Starting with its first adjacent node, find the next node that forms the smallest angle with the previous line
 - Repeating until original node reached again
 - Using more loops to get rid of the duplicates
 - Using even more loops to split them into triangles and quadrilaterals



Implementation: Modular triangle creases



Finding creases – lots of maths:

- An active triangle only has 3 creases – the lines from the corners to the incenter
- An inactive triangle is more tricky – the extra highlighted lines are needed
 - Point A: where two circles tangent on the active edge (one of)
 - Point B: line extended from A at right angle to active edge to intersect with angle bisector of active and inactive edges
 - Point C: Intersection between extended line from B and inactive edge at right angle
 - AB and BC are creases



Modular quad

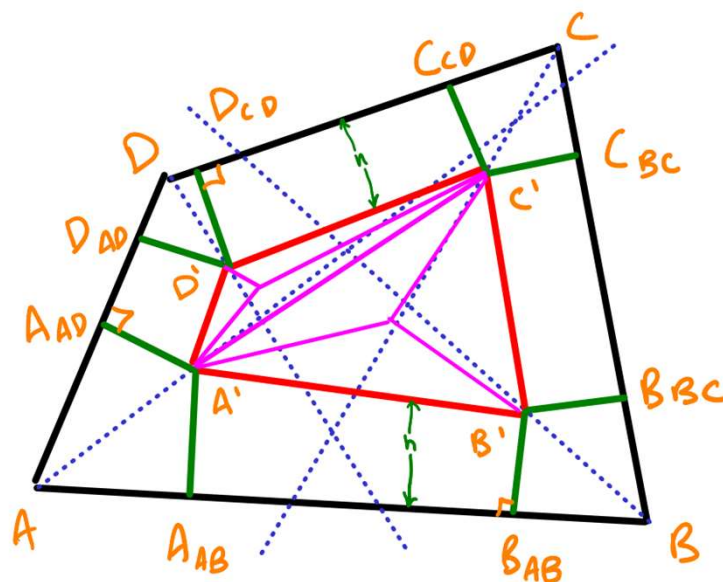
Finding creases – lots of maths:

- Finding an inset quadrilateral $A'B'C'D'$ where the edges are equidistant from the outer quadrilateral, at a distance h
- At least one of two equations must hold for unique h :

$$(i) \quad |AA_{AB}| + |A'C'| + |CC_{CB}| = |AC|$$

$$(ii) \quad |DD_{DA}| + |D'B'| + |BB_{BC}| = |DB|$$

- Only one equation holds: the inset points collapse into a line
- None of the equations hold: inset points collapse into a unique point



ORIGAMI

ORIGAMI



Showcase

ORIGAMI



Node Properties

X:

Y:

Edge Lengths



Comparison with the official TreeMaker: Menu

Project results

Node Properties

X: 245
Y: 253.4583282470703

Update Node
Scale Everything
Generate Crease Pattern
View Creases Only

Edge Lengths

- Edge ID 0: 1
- Edge ID 1: 1
- Edge ID 2: 3
- Edge ID 3: 1
- Edge ID 4: 1

Conclusions

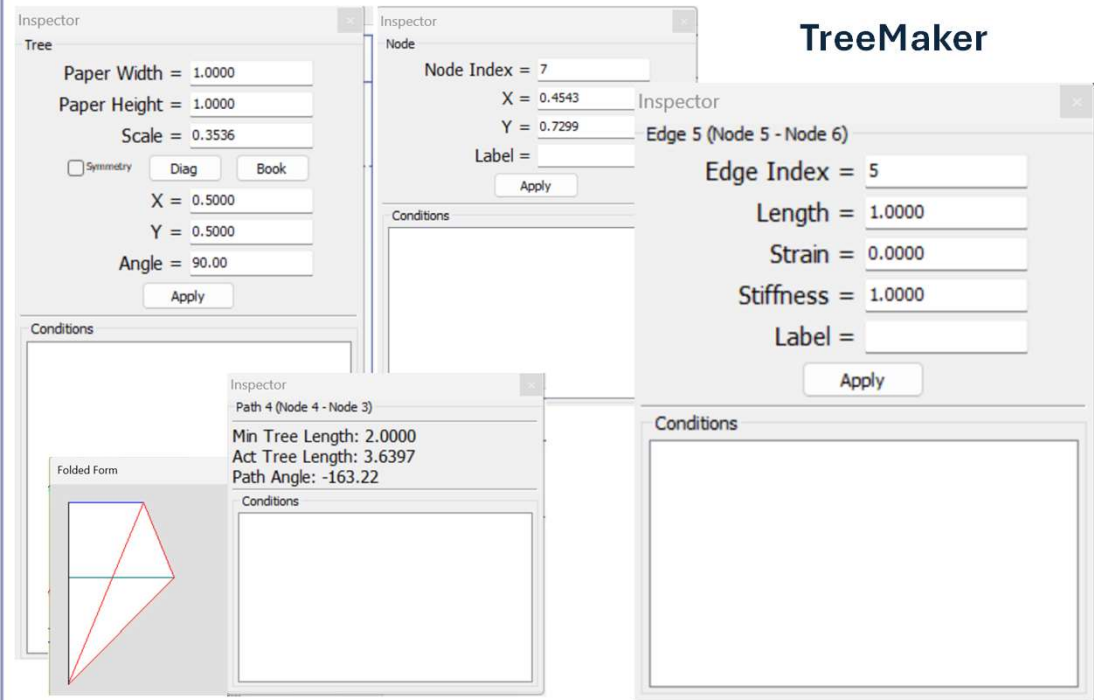
A lot is lacking

Only variables:

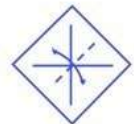
- Edge lengths
- Node coordinates
- Viewing creases
- Scaling

No folded form

No Stiffness attribute

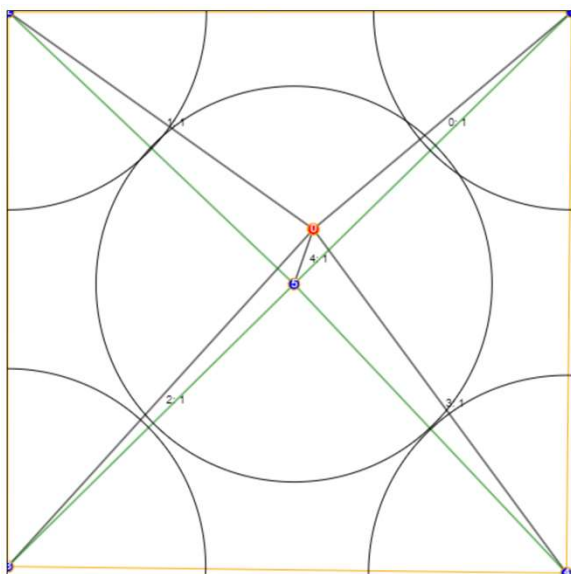


O R I G A M I



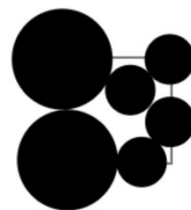
Comparison with the official TreeMaker: Circle Packing

Project results

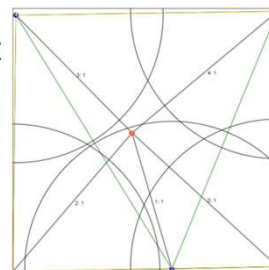


Conclusions

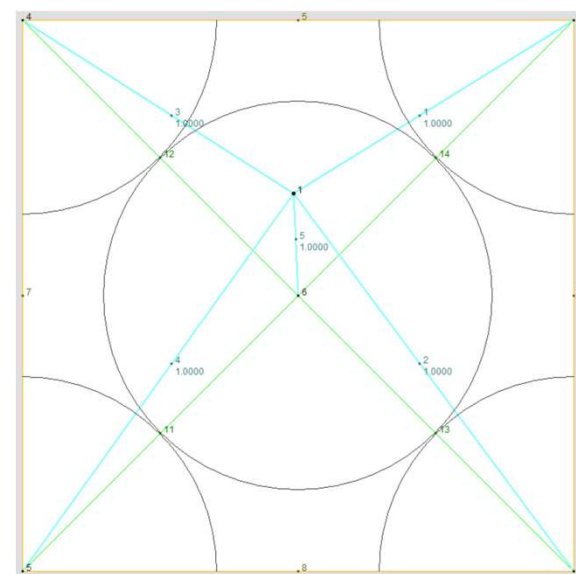
- Works with circles of varying radii
- Integers only – less accurate so some overlaps



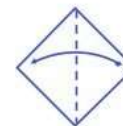
Physics doesn't always work:



TreeMaker

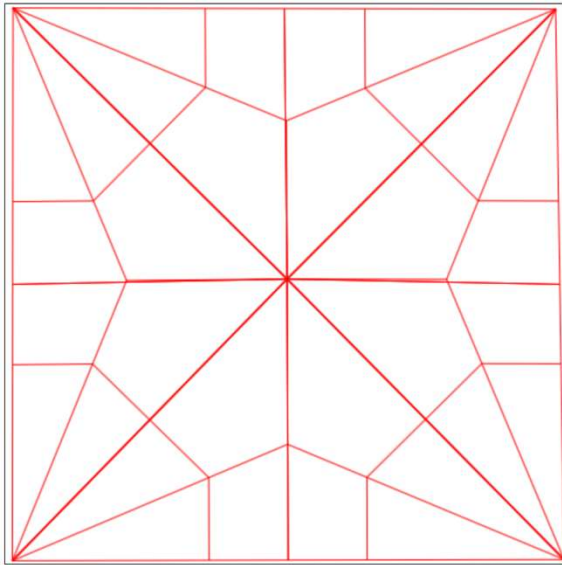


O R I G A M I



Comparison with the official TreeMaker: Triangle creases

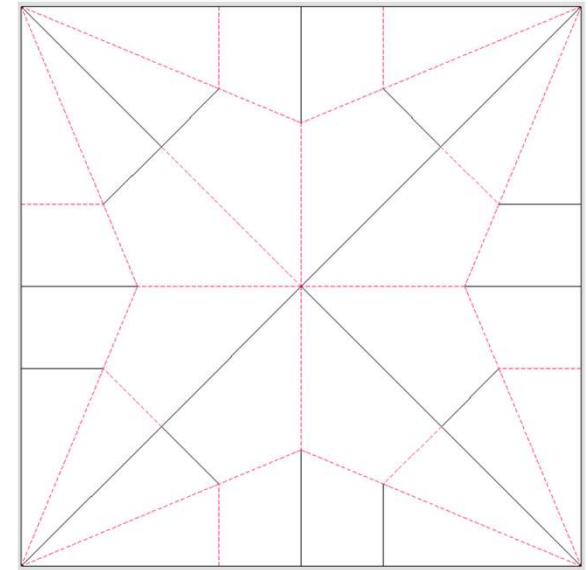
Project results



Conclusions

- Geometry and maths working in order
- Less accurate due to circle packing errors
- No differentiation between mountain and valley folds

TreeMaker

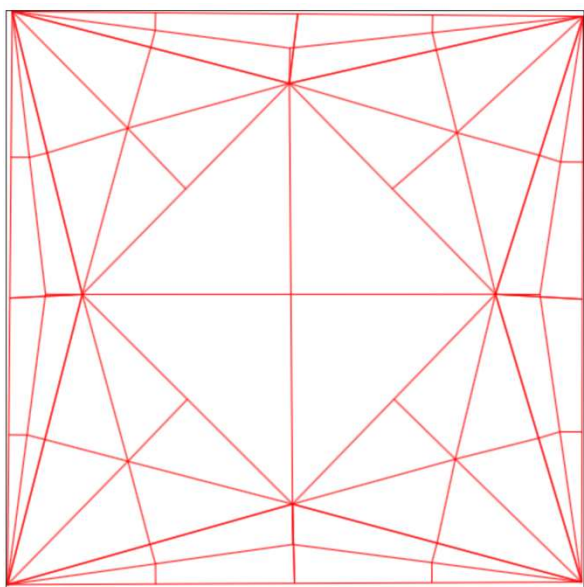


O R I G A M I



Comparison with the official TreeMaker: Quad creases

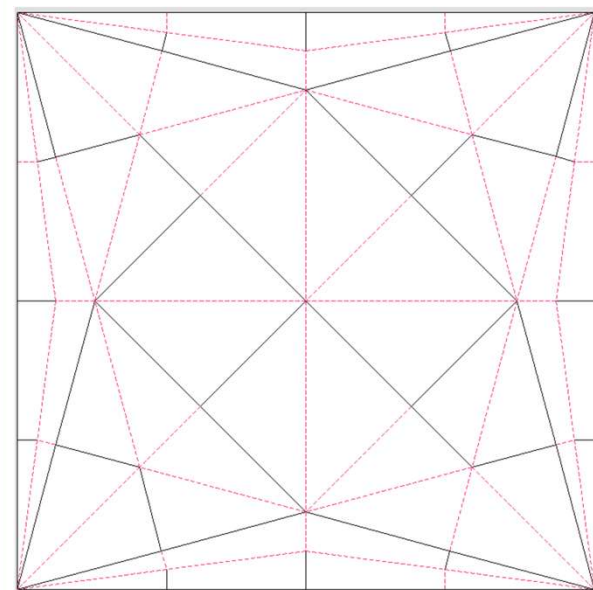
Project results



Conclusions

- Disclaimer: Lost code due to blue screen
- Temporary fix was to use only return the Huisimi quad creases if it's possible (as shows on left, lacking a few bits)
- Full Lang's gusset quad not implemented

TreeMaker

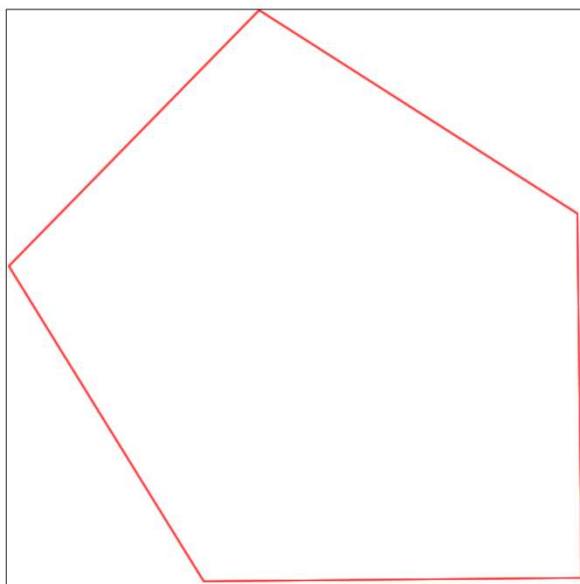


O R I G A M I



Comparison with the official TreeMaker: Large polygons

Project results



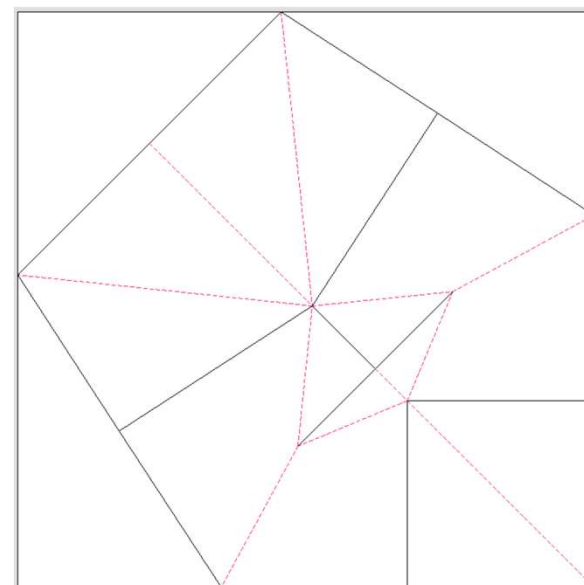
Conclusions

- Not implemented
- TreeMaker uses an algorithm for universal molecules similar to quad (with angle bisectors and h and so forth)

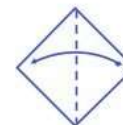
For future:

- Understanding the theory behind the universal molecule

TreeMaker



O R I G A M I

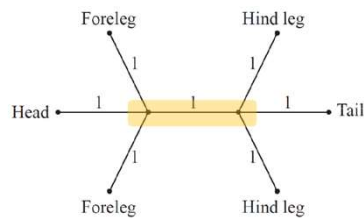


-
-
-

- Not implemented

For future:

- Extending on the “fake” physics circle packing, could use a flexible rectangle to represent a river



ORIGAMI



ORIGAMI



**Thankyou! Happy
folding**

